

Distributed Concurrency Control & Isolation Levels

Complete Study Notes

Dirty Read · Non-Repeatable Read · Phantom Read · Isolation Levels
Optimistic CC · Pessimistic CC · Deadlocks · Locking Strategies

TOPICS COVERED

1. Race Conditions & Why Concurrency Control

2. Transactions & ACID in Concurrency

3. Shared vs Exclusive Locks

4. Isolation Levels (0-3)

5. Optimistic Concurrency Control

6. Pessimistic Concurrency Control

7. Deadlocks — Detection & Resolution

8. Quick Revision & Interview Questions

1. Race Conditions & The Concurrency Problem	3
2. Transactions & ACID in Concurrency	3
3. Shared vs Exclusive Locks	3
4. Three Read Anomalies — Dirty, Non-Repeatable, Phantom	4
5. Transaction Isolation Levels (0-3)	5
6. Optimistic Concurrency Control	6
7. Pessimistic Concurrency Control	6
8. Deadlocks — Detection, Resolution, Prevention	7
9. Quick Revision & Interview Questions	8

1, 2 & 3. Race Conditions, Transactions & Locking

Concurrency problem: Two users simultaneously book Seat A1 at BookMyShow. Both read the seat as AVAILABLE at the same moment. Without concurrency control, both proceed — one seat gets double-booked. This is a **race condition** on a shared resource.

Scenario	Single Machine	Distributed System
Multiple threads	synchronized block works — one thread at a time	synchronized does NOT work — different processes/machines
Solution	Thread-level mutex/lock	Database-level locking + isolation levels
Mechanism	OS scheduler enforces single-thread access	DB acquires locks on rows; isolation controls visibility

Shared (S-Lock) vs Exclusive (X-Lock)

Lock Type	Symbol	Who can hold it	Blocks whom	Used for
Shared Lock	S	Multiple transactions simultaneously	Blocks X-lock acquisition	Reads — SELECT
Exclusive Lock	X	Only one transaction at a time	Blocks ALL other S and X locks	Writes — INSERT/UPDATE/DELETE

$S + S = \text{allowed}$. Two transactions can both read the same row simultaneously.

$S + X = \text{blocked}$. If T1 holds S-lock, T2 cannot acquire X-lock (cannot write).

$X + X = \text{blocked}$. Only one writer at a time. All others wait.

Lock granularity: row-level (most common), page-level, table-level. Row-level = highest concurrency.

4. The Three Read Anomalies

Three classic problems arise when transactions run concurrently without sufficient isolation. Each isolation level is designed to prevent a specific set of these anomalies.

Concurrency Problems — Dirty Read, Non-Repeatable Read, Phantom Read

Dirty Read T1 reads uncommitted data from T2 T2 BEGIN — UPDATE seat=BOOKED write, not yet committed T1 SELECT seat ← sees BOOKED! reads T2 uncommitted data T2 ROLLBACK — seat reverts T2 rolls back T1 Proceeds with wrong data data was never real! Problem: T1 acted on data that was never committed — invalid decision	Non-Repeatable Read T1 reads same row twice, gets different values T1 SELECT price = 100 first read of row T2 UPDATE price = 150 T2 commits change T1 SELECT price = 150 second read: different! T1 Inconsistency within tx same query, different result Problem: Same row, same transaction, different values on re-read	Phantom Read T1 re-runs query, gets different set of rows T1 SELECT * WHERE dept=Eng returns 10 rows T2 INSERT new Eng employee T2 commits new row T1 SELECT * WHERE dept=Eng returns 11 rows! T1 New row appeared mid-tx phantom appeared Problem: Row count changes between two identical queries in same transaction
---	---	---

Dirty Read: uncommitted data read · Non-Repeatable Read: same row, different value on re-read · Phantom Read: different row count on re-query

Anomaly	What happens	Example	Prevented by
Dirty Read	T1 reads T2's uncommitted change. T2 rolls back — T1 acted on invalid data.	T1 reads seat=BOOKED before T2 commits. T2 rolls back — seat was never booked.	Read Committed (Level 1) and above
Non-Repeatable Read	T1 reads row twice. T2 commits a change between the two reads.	T1 reads price=100, T2 updates to 150, T1 reads again: 150. Different value.	Repeatable Read (Level 2) and above
Phantom Read	T1 runs same query twice. T2 inserts/deletes rows between runs.	T1 queries dept=Eng: 10 rows. T2 inserts new Eng employee. T1 re-queries: 11 rows.	Serializable (Level 3) only

5. Transaction Isolation Levels (0 to 3)

SQL standard defines 4 isolation levels. Higher level = stronger consistency = lower concurrency. The right level depends on your application's tolerance for anomalies vs need for throughput.

Transaction Isolation Levels — Locking Strategy Anomalies Prevented

Isolation Level	Locking	Dirty	Non-Rep	Phantom	Use Case
0 — Read Uncommitted No locks at all	Reads: no lock Writes: no lock Held: none	YES	YES	YES	Rarely used. Analytics on non-critical data only.
1 — Read Committed S-lock released immediately after read	Reads: S-lock (released instantly) Writes: X-lock till commit Held: instant release	NO	YES	YES	Most DB defaults (PostgreSQL). Prevents dirty reads.
2 — Repeatable Read S-lock held until transaction commits	Reads: S-lock held till commit Writes: X-lock till commit Held: till end	NO	NO	YES	MySQL InnoDB default. Used in pessimistic control.
3 — Serializable Range locks prevent any phantom rows	Reads: S-lock + range lock Writes: X-lock till commit Held: till end	NO	NO	NO	Strongest. Simulates serial execution

S-lock = Shared (read lock, multiple allowed) | X-lock = Exclusive (write lock, blocks all)

4 levels: Read Uncommitted → Read Committed → Repeatable Read → Serializable · Each prevents one more anomaly

PostgreSQL default: Read Committed (Level 1). MySQL InnoDB default: Repeatable Read (Level 2).

Most apps use Read Committed — prevents dirty reads, acceptable non-repeatable read risk.

Range lock (Serializable): locks not just the matching rows but also the GAP between them — prevents phantom inserts.

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE — can set per-transaction in any RDBMS.

6 & 7. Optimistic vs Pessimistic Concurrency Control

Both approaches solve concurrent write conflicts but with opposite philosophies. **Pessimistic** assumes conflicts **WILL** happen — lock early, prevent conflict. **Optimistic** assumes conflicts are **RARE** — read freely, validate at write time.

Pessimistic vs Optimistic Concurrency Control

Pessimistic Concurrency Control	Optimistic Concurrency Control
Lock first, ask questions later	Read freely, validate before write
T1 starts acquires X-lock on row	T1 reads row gets row + version=5
T2 tries read BLOCKED — waits for T1 lock	T2 reads row gets row + version=5
T1 writes holds lock, makes changes	T1 writes checks version=5 ✓ → commit, v=6
T1 commits lock released — T2 unblocks	T2 writes checks version: 5≠6 → RETRY
Deadlock Risk: T1 holds A, wants B T2 holds B, wants A — circular wait! Isolation level: Repeatable Read / Serializable Lock duration: Held for full transaction Deadlocks: Possible — needs detection Throughput: Lower — threads blocked	Version Check (Row Versioning): Read row + version number Before write: version match? commit : retry Isolation level: Read Committed Lock duration: Short — write phase only Deadlocks: None — no long-held locks Throughput: Higher — no blocking reads
Interview rule: High contention (many conflicts) = Pessimistic. Low contention (rare conflicts) = Optimistic. Real world: Banking = Pessimistic. E-commerce product catalog reads = Optimistic. Booking (BookMyShow) = Pessimistic for seats.	

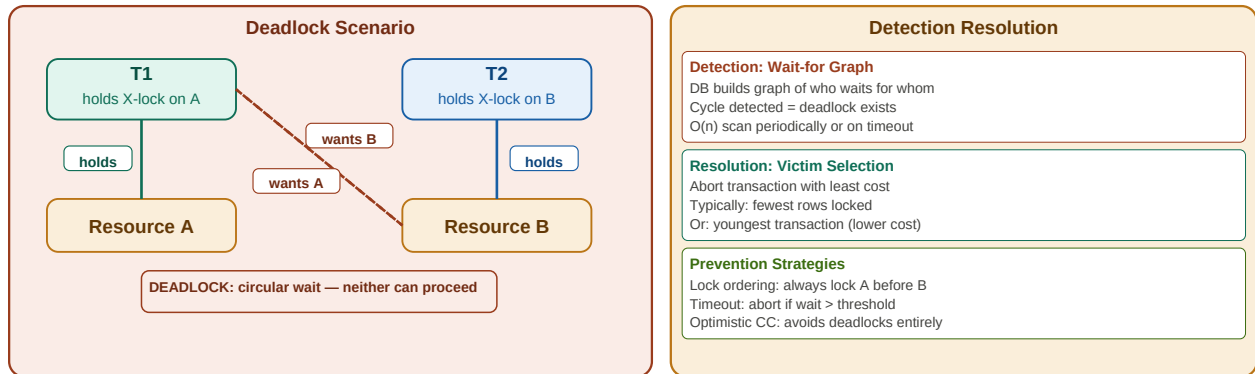
Pessimistic: lock for full duration, blocks others · Optimistic: version check before commit, retry on conflict · No deadlocks in OCC

Feature	Optimistic CC	Pessimistic CC
Isolation Level Used	Read Committed	Repeatable Read / Serializable
Lock Duration	Short — write phase only	Full transaction duration
Deadlock Possibility	None — no long locks held	Yes — circular wait possible
Conflict Detection	Version/timestamp mismatch at commit	Lock wait / blocking
On Conflict	Abort + retry (application logic)	Wait for lock release or abort deadlock victim
Throughput	Higher — reads never block	Lower — blocked threads accumulate
Suitable For	Read-heavy, low-contention (catalog reads)	High-contention (seat booking, banking)
Real Example	Amazon product page views	BookMyShow seat reservation, bank transfer

8. Deadlocks — Detection, Resolution & Prevention

A **deadlock** occurs when two or more transactions each hold a lock that another transaction needs, creating a circular wait. Neither can proceed. This is unique to pessimistic concurrency control.

Deadlock — How it Happens How to Resolve It



Key insight: Optimistic CC avoids deadlocks entirely since it never holds long-lived locks.
 Pessimistic CC needs deadlock detection + victim selection as part of its design.

T1 holds A wants B · T2 holds B wants A → circular wait → deadlock · Resolution: abort one victim

Deadlock Handling Strategies

Strategy	How it works	Tradeoff
Wait-for graph detection	DB builds directed graph: T1 waits for T2, T2 waits for T1 = cycle = deadlock	Overhead of periodic graph scan
Timeout	If lock wait exceeds threshold, abort transaction	May abort non-deadlocked transactions
Lock ordering	Always acquire locks in same order (e.g. always A before B)	Requires design discipline across all code paths
Optimistic CC	No long locks = no circular waits = no deadlocks possible	Retry overhead on conflicts

Victim selection: DB aborts the transaction with the lowest cost — typically fewest rows locked or youngest transaction.

InnoDB deadlock detection: runs a background thread, detects cycles in the lock wait graph, aborts victim within milliseconds.

Application responsibility: always retry transactions that fail due to deadlock — most ORMs do this automatically.

Two-Phase Locking (2PL): formal pessimistic protocol. Growing phase: acquire locks. Shrinking phase: release locks. No lock acquired after first release.

9. Quick Revision & Interview Questions

Quick Revision

- > Concurrency problem: multiple transactions access same resource simultaneously → race condition → inconsistent state.
- > synchronized block works in single-process multithreading only. Does NOT work in distributed systems.
- > Distributed solution: database-level locking + transaction isolation levels.
- > S-lock (Shared): multiple readers allowed simultaneously. Blocks X-lock (writers).
- > X-lock (Exclusive): only one holder. Blocks ALL other reads and writes.
- > Dirty Read: reading uncommitted data. Fixed at Read Committed (Level 1).
- > Non-Repeatable Read: same row, different value on re-read. Fixed at Repeatable Read (Level 2).
- > Phantom Read: different row count on re-query (inserts/deletes). Fixed at Serializable (Level 3).
- > Read Uncommitted (0): no locks. All anomalies possible.
- > Read Committed (1): S-lock released immediately. Prevents dirty reads. PostgreSQL default.
- > Repeatable Read (2): S-lock held till commit. Prevents dirty + non-repeatable. MySQL default.
- > Serializable (3): range locks. Prevents all anomalies. Lowest concurrency.
- > Pessimistic CC: lock early, hold for full transaction. Uses Repeatable Read / Serializable.
- > Optimistic CC: read freely, validate with version check before write. No deadlocks. Uses Read Committed.
- > Deadlock: T1 holds A wants B, T2 holds B wants A → circular wait → neither proceeds.
- > Deadlock resolution: abort victim (lowest cost transaction). Application retries.
- > Deadlock prevention: lock ordering (always same sequence), timeout, or use Optimistic CC.
- > Two-Phase Locking (2PL): Growing phase (acquire locks) then Shrinking phase (release). No reacquire.

Interview Questions

- > 1. What is a race condition? How does it apply to database systems?
- > 2. Why does synchronized keyword not work for distributed concurrency control?
- > 3. What is the difference between a shared lock and an exclusive lock?
- > 4. Explain dirty read, non-repeatable read, and phantom read with examples.
- > 5. Walk me through the 4 isolation levels and what each one prevents.
- > 6. What is the difference between Read Committed and Repeatable Read?
- > 7. How does Serializable isolation prevent phantom reads?
- > 8. Explain Optimistic Concurrency Control. How does version checking work?
- > 9. Explain Pessimistic Concurrency Control. When would you choose it?
- > 10. When would you use Optimistic vs Pessimistic CC? Give real-world examples.
- > 11. What is a deadlock? Give a concrete example with two transactions.
- > 12. How does a database detect and resolve deadlocks?
- > 13. What is Two-Phase Locking (2PL)? What are its two phases?
- > 14. BookMyShow seat booking — which concurrency control strategy and isolation level?
- > 15. What is the default isolation level in PostgreSQL? In MySQL InnoDB?

Key Terms

Race Condition	Multiple threads/processes access shared resource simultaneously → inconsistent state
S-Lock (Shared)	Read lock. Multiple held concurrently. Blocks X-lock acquisition.
X-Lock (Exclusive)	Write lock. Only one held. Blocks all other S and X locks.
Dirty Read	Reading uncommitted changes of another transaction that may rollback
Non-Repeatable Read	Re-reading same row in one transaction yields different values
Phantom Read	Re-running a query yields different set of rows (inserts/deletes by others)
Read Uncommitted	Level 0: no locks. All three anomalies possible.
Read Committed	Level 1: S-lock released after read. Prevents dirty reads. PG default.
Repeatable Read	Level 2: S-lock held till commit. Prevents dirty + non-repeatable. MySQL default.
Serializable	Level 3: range locks. Prevents all anomalies. Lowest concurrency.
Optimistic CC	Read freely + version check before write. No deadlocks. Read Committed.
Pessimistic CC	Long-held locks. Prevents conflicts. Deadlocks possible. Repeatable Read.
Deadlock	Circular lock wait between two transactions. Neither can proceed.
2PL	Two-Phase Locking: grow phase (acquire) then shrink phase (release). No reacquire.
Range Lock	Lock on a range of rows + gaps. Used in Serializable to prevent phantom reads.